

CM500326

Foundations of Artificial Intelligence

Graded Assignment 1: Sudoku

Dr Nadejda Roubtsova and Djordje Bozic

Due	Wednesday, 30 September 2026 before 12:00 midday (UK time)
Percentage of overall unit mark	50%
Submission Components	python implementation and report
Submission Format	jupyter notebook <code>.ipynb</code> (sudoku.ipynb) text document <code>.pdf</code> (report.pdf)
Anonymous Marking	Yes
Generative AI Assessment Categorisation	Closed Lane (guidance)

1 Overview

The sudoku coursework assignment consists of two tasks:

In **Task 1 (solver implementation)**, you will implement from scratch an AI agent to solve sudoku puzzles using any algorithm of your choice. You need to only consider 9×9 sudoku grids where each sudoku has either a single solution or no solution.

In **Task 2 (report)**, you will write a 1-page report to explain and discuss your solution.

2 Task description and deliverables

Task 1 (solver implementation) is worth 80 points (80% of the coursework mark). You will be given 1) a starter notebook `sudoku.ipynb` detailing the required implementation template and 2) a set of sudoku puzzles for testing. You will write and validate your solver implementation in the notebook provided following the instructions in it, crucially including those on ensuring the compatibility with the automated testing script.

Important note: We test all submissions with **numpy 1.26.4** and **python 3.10** as default. If you do not use the same configuration, the performance of your solver can be negatively affected during testing. To avoid that, please ensure you use the stated numpy and python versions to write your implementation (or do get in touch with us well in advance of submission in case of any insurmountable problems with this requirement).

The choice of algorithm to solve sudoku puzzles is up to you, provided you can implement it yourself from scratch. We expect you will use search algorithms taught in the unit, but you are welcome to implement your own ideas or do independent research for suitable approaches. There is often a trade-off between performance and complexity of implementation.

You could for example implement an algorithm that treats sudoku as a *constraint satisfaction problem* (CSP) as it is taught in the unit. You may use any relevant code you have previously worked with in ungraded practicals of this unit as a guide for your implementation. A good implementation of backtracking depth-first search for CSPs with constraint propagation would make a strong submission. There are also both easier and more complex options for your solver approach.

Implementation of only one algorithm is required and will be assessed as your submission in the automated testing. If you experiment with more than one algorithm, please ensure that you submit only your best performing one at the end.

You are encouraged to implement your chosen algorithm efficiently, using standard python. However, this assignment is **not** about multi-threading, use of accelerated python or the GPU. Such methods to gain a speed advantage will not be accepted, and you may get a zero if such an unauthorised enhancement cannot be easily disabled in your code before testing. Your solution must be designed to run in the default single-core jupyter notebook environment.

The deliverable for Task 1 (solver implementation) is the `sudoku.ipynb` jupyter notebook with your solver implementation using python 3.10 and numpy 1.26.4, validated using pre-submission tests detailed in the notebook.

Task 2 (report) is worth 20 points (20% of the coursework mark). For this task, you need to write a 1-page report to present and discuss your implemented solver solution. The page limit is strict, but excludes references.

The report must consist of the following sections:

- **Statement of approach:** Using proper terminology, provide a statement of the approach you have taken to solve the task.
- **Core algorithm description:** A description of the core algorithm you have implemented in technically sound terminology and, where appropriate, formalised with supporting mathematical notation and/or diagrams.
- **Optimisations:** A statement of all employed optimisations to the core algorithm and/or its basic implementation. Highlight the contribution of each optimisation to reducing the overall search complexity (your quantitative results must reflect the claims made).
- **Reflections and suggestions for further work:** Provide an assessment of your solver's performance with reference to suitable quantitative and qualitative metrics. Both strengths and limitations should be part of the assessment. Propose any future work that could address the current limitations.

The deliverable for Task 2 (report) is the text file `report.pdf`.

3 Marking Criteria

Task 1 (solver implementation) will be assessed using automated testing based on the key criteria of correctness and speed of sudoku puzzle solving. The automarker will use a set of test sudoku puzzles of various difficulty, some of which are in your given data (*provided set*) but most of which are not (*hidden set*).

It is recommended that you ensure correctness of your solver for all in-scope scenarios before you start optimising it for the speed criterion. Incorrectly solved puzzles will be penalised more by the automarking script, than the ones which time out.

For the speed criterion, the simple guideline is *the faster, the better*. The automarker operates with an elaborate system of timeouts (both for each puzzle and each difficulty level) and failure limits (both time and correctness-wise), as it runs your solver on the test set of puzzles. The grade is determined by how far your solver gets through this obstacle course. The script begins testing from the *very easy* category up to the *very hard* category.

The absolute upper bound on solving time is 30 seconds i.e., the automarker will never give any credit for a puzzle taking longer than that. If you consistently get per-puzzle latencies close to the upper limit, you should re-consider your approach. On the other hand, you know you are on the right track to a good implementation if you are getting an average latency of under one second per puzzle. There will naturally be differences in average latencies for different difficulty levels: a fairly standard solver implementation may get 0.001 seconds per puzzle for the *very easy* sudoku category, but struggle to finish under the upper bound of 30 seconds for the *hard* sudokus.

Finally, please be mindful that the measured latencies will vary with hardware. The automarker will be run on the same test machine to ensure consistent relative performance of all submissions and grade fairness. However, the absolute speeds we measure on the test machine may not necessarily be the same as observed on your machine: for example, if yours is a particularly advanced hardware one, you may get a favourably misrepresented speed performance relative to our test machine. The safeguard against falling into the trap is 1) to have an objective theory-based expectation of the performance of your approach and 2) not to stop optimising it too prematurely based just on the absolute speeds observed.

Important: working from scratch. Please note that the assignment explicitly states that your implementation must be **from scratch**. Anyone found by the marker not to be working from scratch but from an existing third-party implementation will have their mark for Task 1 capped 40/80 (50%). That holds even if the source of the third-party implementation is acknowledged (please also see Section 5 for a related discussion on academic integrity).

Task 2 (report) will be assessed out of 20 points, with each section outlined in Section 2 (under Task 2 (report)) being 5% of the coursework mark or 5 points.

The following are the indicative marking criteria of the report:

Statement of approach (5 points / 5%)

- The objective difficulty of coherently stated approaches will be assessed and credited by the marker;
- Marks will be deducted for incorrect use of terminology in the statement of approach;
- Marks will be deducted if there is an inconsistency between what is claimed to be the approach in the report and the observed performance of the solver.

Core algorithm description (5 points / 5%)

- How well understood and explained is the algorithm that has been implemented?
- Is the explanation suitably conceptually generalised or, on the contrary, too low-level, detailing the algorithm in terms of the functions of own implementation?
- Is the core algorithm explained separately from the optimisations?

Optimisations (5 points / 5%)

- How well have the optimisations been explained?
- How sophisticated and numerous have the optimisations been?
- How successful have the optimisations been? Highlight the contribution of each optimisation to reducing the overall search complexity, both qualitatively and with reference to your empirical solver performance results. If you attempt formal theoretical complexity reduction analysis using complexity measures for bonus credit, make sure the statements are fully mathematically formalised and derived in the report: quantitative statements on complexity metrics of optimised solutions quoted without a full derivation will not be credited.

Reflections and suggestions for further work (5 points / 5%)

- How well has the success of your approach been evaluated?
- Have the strengths and limitations been explicitly identified?
- Has sensible and interesting future work been proposed to address *specific* limitations?
- Have the future work methods proposed been conceptually defined?

The following penalties will be applied if the report is over the page limit:

- -2 points for going slightly (a couple of lines) over the 1-page limit;
- -5 points for going significantly (anything above a few lines) over the 1-page limit;
- No content beyond the second page will be read by the marker at all.

Please also see Q2 in the FAQs (Section 6) regarding document formatting and penalties for choosing an unreasonable format to squeeze more text in.

The bibliography (your list of references at the end) is excluded from the page count. Please make it as long as it needs to be to fully acknowledge all sources used.

4 Feedback and Getting Help

For **formative guidance** and further **assignment clarifications**, you may post generic questions on the Engage Q&A forum for the unit convenors to answer. Please be mindful about academic integrity (see Section 5) and don't post specifics of your implementation.

You will receive **summative feedback** on your work together with the coursework mark within 15 working days of the submission deadline.

5 Academic Integrity

Coursework is designed to help improve your understanding first and foremost – the process of doing the assignment is part of *learning*. It is also used to assess your ability, and so you must uphold academic integrity. Submitting plagiarised work risks your entire place on your degree.

Do not work with (or copy work from) other students for the coursework assignment. In the case of substantial submission similarities, *everyone* involved will have to go through a plagiarism investigation.

Let us highlight the key points on academic integrity for each task of the coursework:

Task 1 (solver implementation) must be fully implemented by you from scratch. Your code will be checked to ensure that you have not plagiarised.

If you are not working from scratch but acknowledge and reference the source of the third-party implementation you are building on, your mark for Task 1 will be capped at 50% as stated in the marking criteria (Section 3). In this case, it is not considered plagiarism but the mark will be capped as one of the key learning objectives is not achieved.

If you are not working from scratch and do not acknowledge and reference the source of the third-party implementation you are building on, you will be referred to the Director of Studies for a full plagiarism investigation and you may end up with a zero for this assignment.

However, just to clarify, you may:

- re-use any code from the ungraded practicals made available in this unit's material without penalty;
- research any suitable algorithm from prior art literature - if you implement it yourself from pseudo-code or a theoretical idea (with referencing), rather than adopting and building on a ready-made implementation, the mark for Task 1 will not be capped.

Task 2 (report) must be written in your own words and from your own understanding.

Remember that the published work that you refer to in your report should be clearly referenced in your text and listed in a bibliography section given at the end of your report. For more information, see <https://library.bath.ac.uk/referencing/new-to-referencing>.

Beware of excessive use of direct quotes from sources in the reports as it is rarely a good idea. If direct quotes are not properly marked as such with quotation marks and a source reference, it is plagiarism. Excessive use of direct quotes with proper referencing shows little own comprehension and will receive little credit.

Use of Generative AI: The entire coursework (both task 1 and 2) is **Closed Lane**, which means that the use of generative AI is strictly prohibited. For more information, see: <https://teachinghub.bath.ac.uk/guide/genai-and-assessment/>.

Any use of GenAI for this assignment, be it coding or writing, prevents attainment of learning objectives and ultimately devalues your degree. It is our expectation that what you submit is fully authored by you. So, you should be aware that submitting GenAI assisted work as fully your own you are likely legally in breach of the terms and conditions of the GenAI tools, which typically expressly prohibit representation of their model outputs as human generated.

Finally, not using GenAI in your work helps avoid your premature cognitive decline (see the recent [MIT study](#)) and develop professional skills beyond trivial prompt engineering.

6 FAQs

Q1: Can I have further details of the automarking script for the solver implementation? What specific average solver speed do I need to achieve per puzzle for each difficulty level for a mark in the pass / merit / distinction range?

A1: We do not reveal the details of the exact mapping of solver performance to a given level of achievement for several reasons. Firstly, the absolute timings can vary with the hardware

of your testing machine, and hence cannot be quoted as a rock-solid benchmark. Secondly, we would like you to optimise for solver speed (*the faster, the better*), not for the mark you would like to have. General indications of whether you are on the right track with the latencies can be found in Section 3 (under Task 1 (solver implementation)).

Q2: I struggle to fit everything I want to say for Task 2 into a one page report. Can I adjust the typesetting of the text document (e.g., font type/size, margin width, spacing between lines/paragraphs etc.) to fit more text in? Or are there prescribed typesetting specifications, which I must adopt for my report?

A2: We do not enforce specific typesetting for the report in Task 2. However, we do ask you to be reasonable with your choices. It is in your interests that the report is legible to the marker: e.g., we would strongly suggest using standard single spacing between the lines and not going below font size 11. We reserve the right to penalise reports abusing the freedom of report typesetting if it affects legibility. Furthermore, please bear in mind that if you feel you need to endlessly struggle for extra space, there is a chance you are including irrelevant detail.

Q3: Do I submit the jupyter notebook with my solver and the report as two separate files or zipped up into a single file?

A3: Please submit them as two separate files (no need to zip them up). The automarker will place the report .pdf into the same folder with your notebook automatically when the submission is unpacked.

Q4: Do I need to submit the test data and the image folder provided with the assignment?

A4: No, there is no need for that. We already have them.

Q5: Can I submit any additional files, other than the jupyter notebook and the report, which are essential to run my solver?

A5: You should not need any additional files as the implementation in the notebook is meant to be self-contained. Please adhere to the coursework specification and plan your solver implementation in such a way that the coursework can be submitted as two files: the jupyter notebook (sudoku.ipynb) and the report (report.pdf) with no additional ones required.

Q6: There have been difficult circumstances affecting my ability to study. Can I get a coursework extension?

A6: Unit conveners cannot grant extensions. The procedure for requesting extensions is detailed on the Programme Hubs on Engage (under *Extension Requests and Study Breaks*).